

Bài Lab này sẽ mô phỏng hoàn chỉnh quá trình **Thêm / Cập nhật thông tin** bằng Form, minh họa luồng truyền dữ liệu (Props & State) và đặc biệt là cách lưu trữ dữ liệu vĩnh viễn trên trình duyệt bằng **LocalStorage** (giải quyết bài toán mất dữ liệu khi F5 (reload) trang mà không cần CSDL thật).

LAB 4: XÂY DỰNG TÍNH NĂNG QUẢN LÝ THÔNG TIN NGƯỜI DÙNG TƯƠNG TÁC FORM VÀ LƯU TRỮ LOCALSTORAGE

Mục tiêu bài Lab

- Hiểu cách ràng buộc dữ liệu từ ô nhập liệu (Input) vào State của React (Controlled Components).
- Thực hành truyền dữ liệu 2 chiều: Cha truyền dữ liệu khởi tạo xuống Form (Con), Form truyền dữ liệu mới lên Cha.
- Làm quen với **LocalStorage** để lưu trữ dữ liệu dưới máy khách, giữ nguyên thông tin kể cả khi tải lại trang web.

Phân tích Cấu trúc Component

Tên Component	Vai trò & Trách nhiệm
ProfileManager (Smart)	Quản lý state userProfile. Đọc/Ghi dữ liệu vào localStorage. Điều phối việc bật/tắt form.
ProfileDisplay (Dumb)	Chỉ nhận dữ liệu userProfile từ Cha để in ra màn hình.
ProfileForm (Dumb)	Chứa các ô input. Nhận dữ liệu cũ để điền sẵn vào form. Gửi dữ liệu mới lên Cha khi bấm "Lưu".

PHẦN 1: TRIỂN KHAI CODE

Bạn hãy tạo một thư mục mới src/features/LocalProfile/ và lần lượt tạo 3 file sau:

Bước 1: Xây dựng Component hiển thị - ProfileDisplay.jsx

Component này làm nhiệm vụ in thông tin. Nếu dữ liệu trống, nó sẽ hiển thị dòng chữ nhắc nhở cập nhật.

```
import React from 'react';
```

LAB 4: XÂY DỰNG TÍNH NĂNG QUẢN LÝ THÔNG TIN NGƯỜI DÙNG(tt)

```
const ProfileDisplay = ({ data, onOpenEdit }) => {
  return (
    <div style={{ border: '1px solid #ddd', padding: '20px', borderRadius: '8px',
      backgroundColor: '#fff' }}>
      <h3 style={{ marginTop: 0 }}>Hồ sơ của tôi</h3>

      {/* Kiểm tra xem đã có tên chưa, nếu chưa tức là hồ sơ trống */}
      {!data.name ? (
        <p style={{ color: '#888', fontStyle: 'italic' }}>
          Bạn chưa có thông tin. Vui lòng cập nhật!
        </p>
      ) : (
        <ul style={{ lineHeight: '1.8', fontSize: '16px' }}>
          <li><strong>Họ và tên:</strong> {data.name}</li>
          <li><strong>Email:</strong> {data.email}</li>
          <li><strong>Số điện thoại:</strong> {data.phone}</li>
          <li><strong>Địa chỉ:</strong> {data.address}</li>
        </ul>
      )}

      <button
        onClick={onOpenEdit}
        style={{ padding: '8px 16px', backgroundColor: '#007bff', color: 'white',
          border: 'none', borderRadius: '4px', cursor: 'pointer', marginTop: '10px' }}
        >
        {data.name ? "Chỉnh sửa thông tin" : "Thêm thông tin"}
      </button>
    </div>
  );
};

export default ProfileDisplay;
```

Bước 2: Xây dựng Form nhập liệu - ProfileForm.jsx

Component này quản lý các ô input. Mỗi khi người dùng gõ phím, nó cập nhật state nội bộ formData.

```
import React, { useState } from 'react';

const ProfileForm = ({ initialData, onSave, onCancel }) => {
  // Dùng dữ liệu Cha truyền xuống để điền sẵn vào Form (hoặc chuỗi rỗng nếu chưa có)
  const [formData, setFormData] = useState({
    name: initialData.name || '',
    email: initialData.email || '',
  });
}
```

```

    phone: initialData.phone || '',
    address: initialData.address || ''
  });

  // Hàm gom chung xử lý khi gõ phím cho mọi ô input
  const handleInputChange = (e) => {
    const { name, value } = e.target;
    setFormData(prevState => ({
      ...prevState,
      [name]: value
    }));
  };

  const handleSubmit = (e) => {
    e.preventDefault(); // Chặn hành vi load lại trang của Form
    onSave(formData); // Gửi cục dữ liệu mới lên cho Cha
  };

  // CSS chung cho input để code gọn hơn
  const inputStyle = { width: '100%', padding: '8px', marginBottom: '15px',
    borderRadius: '4px', border: '1px solid #ccc', boxSizing: 'border-box' };

  return (
    <div style={{ border: '1px solid #ddd', padding: '20px', borderRadius: '8px',
      backgroundColor: '#f9f9f9' }}>
      <h3 style={{ marginTop: 0 }}>Cập nhật thông tin</h3>
      <form onSubmit={handleSubmit}>
        <label>Họ và tên:</label>
        <input name="name" value={formData.name} onChange={handleInputChange}
          style={inputStyle} required />

        <label>Email:</label>
        <input type="email" name="email" value={formData.email}
          onChange={handleInputChange} style={inputStyle} required />

        <label>Số điện thoại:</label>
        <input name="phone" value={formData.phone} onChange={handleInputChange}
          style={inputStyle} required />

        <label>Địa chỉ:</label>
        <input name="address" value={formData.address} onChange={handleInputChange}
          style={inputStyle} required />
      </form>
    </div>
  );

```

LAB 4: XÂY DỰNG TÍNH NĂNG QUẢN LÝ THÔNG TIN NGƯỜI DÙNG(tt)

```
        <button type="submit" style={{ padding: '8px 16px', backgroundColor:
'#28a745', color: 'white', border: 'none', borderRadius: '4px', cursor: 'pointer',
marginRight: '10px' }}>
            Lưu lại
        </button>
        <button type="button" onClick={onCancel} style={{ padding: '8px 16px',
backgroundColor: '#dc3545', color: 'white', border: 'none', borderRadius: '4px',
cursor: 'pointer' }}>
            Hủy bỏ
        </button>
    </form>
</div>
);
};

export default ProfileForm;
```

Bước 3: Xây dựng Smart Component - ProfileManager.jsx

Đây là nơi lưu State gốc và giao tiếp với LocalStorage.

```
import React, { useState, useEffect } from 'react';
import ProfileDisplay from './ProfileDisplay';
import ProfileForm from './ProfileForm';

const ProfileManager = () => {
    // 1. Khởi tạo State bằng cách đọc từ LocalStorage
    const [userProfile, setUserProfile] = useState(() => {
        const savedData = localStorage.getItem('my_profile');
        if (savedData) {
            return JSON.parse(savedData); // Nếu có data cũ, chuyển JSON thành Object
        }
        // Nếu chưa có, trả về object rỗng
        return { name: '', email: '', phone: '', address: '' };
    });

    // State quản lý việc ẩn/hiện Form
    const [isEditing, setIsEditing] = useState(false);

    // 2. Lắng nghe thay đổi: Mỗi khi userProfile thay đổi, tự động lưu vào
    // LocalStorage
    useEffect(() => {
        localStorage.setItem('my_profile', JSON.stringify(userProfile));
    }, [userProfile]); // Dependency array: chỉ chạy khi userProfile thay đổi

    // 3. Hàm nhận dữ liệu mới từ Form con truyền lên
```

```
const handleSaveData = (newData) => {
  setUserProfile(newData); // Cập nhật state gốc
  setIsEditing(false);     // Tắt form
  alert("Lưu thông tin thành công!");
};

return (
  <div style={{ maxWidth: '600px', margin: '40px auto', fontFamily: 'sans-serif' }}>
    <h2 style={{ textAlign: 'center' }}>Thiết lập Tài khoản</h2>

    {/* 4. Điều hướng hiển thị */}
    {isEditing ? (
      <ProfileForm
        initialData={userProfile}
        onSave={handleSaveData}
        onCancel={() => setIsEditing(false)}
      />
    ) : (
      <ProfileDisplay
        data={userProfile}
        onOpenEdit={() => setIsEditing(true)}
      />
    )}
  </div>
);
};

export default ProfileManager;
```

PHẦN 2: HƯỚNG DẪN KIỂM THỬ THỰC TẾ TRÊN LỚP

Sau khi gọi `<ProfileManager/>` vào `App.jsx` và chạy server, bạn hãy hướng dẫn học viên thực hiện chuỗi thao tác sau để thấy rõ "phép thuật" của `LocalStorage`:

Bước 1: Trạng thái ban đầu

- Khi mới mở lên, màn hình sẽ báo *"Bạn chưa có thông tin. Vui lòng cập nhật!"* (Vì `LocalStorage` đang trống).

Bước 2: Thêm thông tin & Kiểm tra luồng Props

- Bấm **"Thêm thông tin"**. Nhập đầy đủ thông tin vào Form.

- Bấm "**Lưu lại**". Giao diện lập tức chuyển về màn hình hiển thị với dữ liệu vừa nhập (Luồng dữ liệu: Form đẩy dữ liệu lên Cha -> Cha cập nhật State -> Cha truyền dữ liệu xuống Display).

Bước 3: Kiểm chứng LocalStorage (Rất quan trọng)

- Yêu cầu học viên bấm F5 (Reload) trình duyệt nhiều lần.
- **Kết quả:** Thông tin vẫn còn nguyên, không bị mất đi như bài Lab trước.
- **Giải thích trực quan:** Hướng dẫn học viên nhấn F12 (mở Developer Tools) -> Chuyển sang tab **Application** (với Chrome) hoặc **Storage** (với Firefox) -> Chọn **Local Storage** ở cột bên trái. Học viên sẽ nhìn thấy một dòng có Key là my_profile và Value là chuỗi JSON chứa thông tin họ vừa nhập.

Bước 4: Sửa và cập nhật

- Bấm "**Chỉnh sửa thông tin**". Form sẽ tự động điền sẵn các thông tin cũ (nhờ truyền initialData xuống).
- Sửa số điện thoại, bấm Lưu. Sau đó mở lại tab Application (F12) để xem chuỗi JSON đã được cập nhật ngay lập tức nhờ useEffect.